

7. class Node:

```
def __init__(self, data):  
    self.data = data  
    self.next = None
```

class Stack:

```
def __init__(self):  
    self.top = None
```

```
def push(self, data):  
    node = Node(data)  
    node.next = self.top  
    self.top = node
```

```
def pop(self):  
    if self.top is None:  
        print("Stack Underflow")  
        return  
  
    print("Popped:", self.top.data)  
    self.top = self.top.next
```

```
def display(self):  
    temp = self.top  
    while temp:  
        print(temp.data)  
        temp = temp.next
```

```
s = Stack()
```

```
while True:
```

```
    print("\n1.Push")
```

```
    print("2.Pop")
```

```
    print("3.Display")
```

```
    print("4.Exit")
```

```
    ch = int(input("Enter choice: "))
```

```
    if ch == 1:
```

```
        s.push(int(input("Enter element: ")))
```

```
    elif ch == 2:
```

```
        s.pop()
```

```
    elif ch == 3:
```

```
        s.display()
```

```
    elif ch == 4:
```

```
        break
```

```
8.class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class Queue:
```

```
def __init__(self):
    self.front = None
    self.rear = None

def insert(self, data):
    node = Node(data)

    if self.rear is None:
        self.front = self.rear = node
        return

    self.rear.next = node
    self.rear = node

def delete(self):
    if self.front is None:
        print("Queue Empty")
        return

    print("Deleted:", self.front.data)
    self.front = self.front.next

    if self.front is None:
        self.rear = None

def display(self):
    temp = self.front
    while temp:
```

```
print(temp.data, end=" ")
temp = temp.next
print()
```

```
q = Queue()
```

```
while True:
```

```
    print("\n1.Insert")
    print("2.Delete")
    print("3.Display")
    print("4.Exit")
```

```
ch = int(input("Enter choice: "))
```

```
if ch == 1:
    q.insert(int(input("Enter element: ")))
elif ch == 2:
    q.delete()
elif ch == 3:
    q.display()
elif ch == 4:
    break
```

```
9. class Node:
```

```
    def __init__(self, data):
        self.data = data
        self.left = None
```

```
self.right = None
```

```
class BST:
```

```
    def __init__(self):
```

```
        self.root = None
```

```
    def insert(self, root, key):
```

```
        if root is None:
```

```
            return Node(key)
```

```
        if key < root.data:
```

```
            root.left = self.insert(root.left, key)
```

```
        else:
```

```
            root.right = self.insert(root.right, key)
```

```
    return root
```

```
    def inorder(self, root):
```

```
        if root:
```

```
            self.inorder(root.left)
```

```
            print(root.data, end=" ")
```

```
            self.inorder(root.right)
```

```
    def preorder(self, root):
```

```
        if root:
```

```
            print(root.data, end=" ")
```

```
            self.preorder(root.left)
```

```
self.preorder(root.right)
```

```
def postorder(self, root):
```

```
    if root:
```

```
        self.postorder(root.left)
```

```
        self.postorder(root.right)
```

```
        print(root.data, end=" ")
```

```
def minimum(self, root):
```

```
    while root.left:
```

```
        root = root.left
```

```
    return root.data
```

```
def maximum(self, root):
```

```
    while root.right:
```

```
        root = root.right
```

```
    return root.data
```

```
def search(self, root, key):
```

```
    if root is None:
```

```
        return False
```

```
    if root.data == key:
```

```
        return True
```

```
    if key < root.data:
```

```
        return self.search(root.left, key)
```

```
return self.search(root.right, key)
```

```
tree = BST()
```

```
while True:
```

```
    print("\n1.Insert")
```

```
    print("2.Inorder")
```

```
    print("3.Preorder")
```

```
    print("4.Postorder")
```

```
    print("5.Maximum")
```

```
    print("6.Minimum")
```

```
    print("7.Search")
```

```
    print("8.Exit")
```

```
ch = int(input("Enter choice: "))
```

```
if ch == 1:
```

```
    tree.root = tree.insert(tree.root, int(input("Enter element: ")))
```

```
elif ch == 2:
```

```
    tree.inorder(tree.root)
```

```
    print()
```

```
elif ch == 3:
```

```
    tree.preorder(tree.root)
```

```
    print()
```

```
elif ch == 4:  
    tree.postorder(tree.root)  
    print()
```

```
elif ch == 5:  
    if tree.root:  
        print("Maximum:", tree.maximum(tree.root))  
    else:  
        print("Tree Empty")
```

```
elif ch == 6:  
    if tree.root:  
        print("Minimum:", tree.minimum(tree.root))  
    else:  
        print("Tree Empty")
```

```
elif ch == 7:  
    key = int(input("Enter element: "))  
    if tree.search(tree.root, key):  
        print("Element Found")  
    else:  
        print("Element Not Found")
```

```
elif ch == 8:  
    break
```